Introduction

Research Tools as Epistemic Infrastructure

Research Tools as Epistemic Infrastructure I

Scientific research operates through a layered ecology of tools, documents, and practices—each shaping what can be known, communicated, and verified. When these layers are fragmented, manuscripts, data, code, and figures can drift out of alignment, creating gaps that are structural rather than incidental. Survey and economic evidence documents the practical cost of failed reproduction attempts (Baker 2016; Freedman et al. 2015), but reproducibility is only the most visible face of a broader problem. Research software engineering, epistemic integrity, and the coordination between human researchers and AI collaborators all depend on whether the tools that produce research artifacts can themselves be transparent, testable, and self-documenting. Nosek et al. (Nosek et al. 2018) argue that committing to analytical plans before data collection is a necessary structural reform; we extend this logic to the research build pipeline.

One root cause is fragmentation—of attention (in the mind) as well as in the cyberphysical niche (documents, versions, and reproducibility artefacts). A typical research project scatters its artifacts across disconnected tools: LaTeX editors (Lamport 1994) for prose, Jupyter notebooks for analysis, ad-hoc shell scripts for figure generation, and manual copy-paste for integrating results into manuscripts. Each boundary between tools is a potential locus of desynchronization. The version of a figure embedded in a PDF may not match the code that ostensibly generated it, while tests may exercise analysis code without exercising the rendering path. Pimentel et al. (Pimentel et al. 2019) studied a corpus described as 1.4 million Jupyter notebooks; in the valid execution subset, 24.11% ran without errors and 4.03% reproduced the recorded results. Those distinctions matter: execution success is

not equivalent to result reproduction. Peng (Peng 2011) argues that reproducibility in computational science requires, at minimum, that the data and code underlying a published result be available for independent verification. Barba (Barba 2018) further documents how "reproducibility," "replicability," and "repeatability" carry conflicting definitions across disciplines, complicating cross-field standards.

Related Work

Related Work I

Gentleman and Temple Lang (Gentleman and Temple Lang 2007) introduced the concept of a *research compendium*—a single unit of scholarly communication bundling code, data, and narrative. This vision has driven two decades of tooling, which can be grouped into four categories: workflow managers, literate programming systems, containerization approaches, and best-practice frameworks.

Snakemake (Köster and Rahmann 2012) uses a rule-based, Python-derived DSL to specify computational workflows as directed acyclic graphs of file-producing steps. It supports containerized execution and an extension architecture for execution and storage backends, while its central scope remains computational pipeline orchestration rather than this repository's combined manuscript, test, and document-provenance contract.

Nextflow (Di Tommaso et al. 2017) employs a dataflow programming paradigm with native support for container-based execution across heterogeneous computing environments (local, SLURM, AWS). Like Snakemake, Nextflow excels at bioinformatics pipeline parallelism but does not address manuscript production, document integrity, or the testing–publication coupling that characterizes research reproducibility.

CWL (Common Workflow Language) (Amstutz et al. 2016) provides a portable, YAML-based standard for describing computational workflows and their dependencies. Its strength lies in interoperability across execution engines (cwltool, Toil, Arvados), but it requires external tooling for manuscript generation and offers no built-in testing or provenance framework.

Knuth's literate programming (Knuth 1984) established the principle that programs should be authored as documents intended for human comprehension. Schulte et al. (Schulte et al. 2012) extended this to multi-language computing environments (Org-mode), demonstrating that literate programming could span languages and output formats.

Quarto (Allaire et al. 2024) extends the R Markdown tradition to support Python, Julia, and Observable, rendering to PDF, HTML, and Word. Quarto integrates code execution with document rendering, achieving a modern form of literate programming, but it does not enforce testing thresholds, manage multi-project repositories, or provide cryptographic provenance.

Jupyter Book (Kluyver et al. 2016) builds on Jupyter notebooks to produce publication-quality documents via Sphinx. While powerful for interactive exploration, Jupyter's notebook format introduces execution-order fragility (Pimentel et al. 2019) and does not naturally support the separation of logic from orchestration that characterizes maintainable research software.

R Markdown (Xie et al. 2018) pioneered knitr-based dynamic documents that weave code and prose. Its ecosystem is rich but R-centric, and it lacks the multi-project management, infrastructure testing, and provenance embedding that characterize `template/`.

Typst (Mädje and Haug 2023) is an emerging markup-based typesetting system with incremental compilation and a programmable scripting layer. While Typst offers faster compilation than LaTeX and a more modern authoring experience, it does not integrate testing, provenance, or multi-project pipeline management.

Docker (Boettiger 2015) addresses reproducibility at the environment level—packaging operating system, libraries, and code into portable containers. While Docker solves the "works on my machine" problem, containerization is complementary to, not a replacement for, the architectural concerns addressed here: Docker does not enforce testing, embed provenance, or manage multi-project manuscript workflows.

Best-Practice Frameworks and Data Standards I

Wilson et al. (Wilson et al. 2017) define "good enough" practices for scientific computing, emphasizing version control, testing, and documentation. Sandve et al. (Sandve et al. 2013) propose ten rules for reproducible computational research. Piccolo and Frampton (Piccolo and Frampton 2016) systematically survey tools for computational reproducibility, finding that environment isolation, workflow automation, and documentation generation address complementary but non-overlapping reproducibility concerns—yet no single tool unifies all three. Stodden et al. (Stodden et al. 2016) advocate for enhanced computational method transparency. The FAIR principles (Wilkinson et al. 2016)—Findable, Accessible, Interoperable, Reusable—establish a standard for data stewardship that has been widely adopted by funding agencies and journals. Lamprecht et al. (Lamprecht et al. 2020)

formalize "Towards FAIR Principles for Research Software," providing the conceptual scaffolding that Barker et al. (Barker et al. 2022) would soon operationalize as the FAIR4RS initiative—recognizing that software has execution, composability, and dependency-management requirements that data-centric FAIR does not address. Cohen et al. (Cohen et al. 2021) characterize the four pillars of research software engineering (software sustainability, software quality, community building, and policy advocacy), situating formal testing and provenance practices within a broader RSE governance framework. Garijo et al. (Garijo et al. 2024) operationalize FAIR4RS through the FAIRsoft evaluator, an automated assessment framework that scores research software against 17+ quality indicators including executability, metadata richness, and documentation completeness. Goble et al. (Goble et al. 2020) extend FAIR to computational workflows specifically,

arguing that workflow provenance requires first-class treatment in scientific computing infrastructure. Nüst et al. (Nüst et al. 2017) introduce the *executable research compendium* (ERC), extending Gentleman and Temple Lang's compendium concept with containerized, interactive reproduction environments. The W3C PROV data model (Moreau and Missier 2013) provides a formal vocabulary for expressing provenance records, while in-toto (Torres-Arias et al. 2019) provides a framework for end-to-end software supply chain integrity verification, and SLSA (Open Source Security Foundation 2023) (Supply-chain Levels for Software Artifacts) extends this to graduated, attestation-based supply-chain security levels for build pipelines. These frameworks articulate *what* reproducible research requires but do not provide an integrated *how*—they lack the tooling, enforcement mechanisms, and architectural patterns that translate standards into practice.

The Gap I

Despite advances in FAIR4RS principles (Barker et al. 2022; Lamprecht et al. 2020), automated FAIR software assessment (Garijo et al. 2024), FAIR computational workflow standards (Goble et al. 2020), supply-chain attestation frameworks (Torres-Arias et al. 2019; Open Source Security Foundation 2023), and the preregistration revolution (Nosek et al. 2018), no existing system integrates six cross-cutting concerns into a single enforced pipeline: (1) end-to-end pipeline orchestration with testing enforcement, (2) multi-format manuscript rendering, (3) cryptographic provenance embedding, (4) multi-project repository management, (5) FAIR-aligned software stewardship, and (6) AI-agent collaboration via structured documentation. Each existing framework addresses a subset; none provides the unified enforcement mechanism. The detailed tool-by-tool comparison is developed in the

The Gap I

Comparison to Existing Tools section; the summary table below captures the gap landscape.

Summary: Gaps Left by Existing Tools I

The six concerns identified above map onto existing tool categories as follows:

Summary: Gaps Left by Existing Tools I

Gap	Partially Addressed By	Not Addressed By
Pipeline orchestration	Snakemake, Nextflow, CWL	Quarto, Jupyter Book, R Markdown, Typst, Overleaf, Prism
Manuscript rendering	Quarto, Jupyter Book, R Markdown, Typst, Overleaf, Prism	Snakemake, Nextflow, CWL, DVC
Testing enforcement	—	All existing tools
Cryptographic provenance	SLSA (build-level attestation only)	All research-focused tools
Multi-project management	—	All existing tools
AI-agent documentation	Overleaf (partial co-author AI), Prism (partial context reasoning)	All pipeline/workflow tools
Agentic skill protocol (MCP-aligned)	—	All existing tools

Summary: Gaps Left by Existing Tools I

No existing system addresses all six concerns within a single enforced pipeline.

`template/`: An Integrated Solution

`template/`: An Integrated Solution I

`template/` was conceived as a structural antidote to this fragmentation. Rather than adding reproducibility as an afterthought—a Docker container wrapping an already-disjointed workflow (Boettiger 2015)—the template enforces integrity at the architectural level. It realizes Gentleman and Temple Lang's research compendium vision (Gentleman and Temple Lang 2007) at repository scale, bundling code, data, tests, manuscripts, and provenance into a single, pipeline-enforced system with version-controlled infrastructure (Ram 2013). It stands on four primary pillars:

`template/`: An Integrated Solution I

1. **Ergonomic Modularity**: A Two-Layer Architecture cleanly separates globally shared infrastructure (logging, rendering, validation, steganography) from project-specific logic (manuscripts, scripts, data). 31 infrastructure subdirectories (28 importable packages) comprising ~1307 Python modules provide reusable services; projects consume them without modification.
2. **Execution Integrity**: Pipeline advancement is contingent on test passage. The configured source-coverage floors are 60% for infrastructure and 90% for each project. The static gate rejects prohibited mock-framework imports, while the suites exercise real filesystem and subprocess behavior and use local protocol servers or explicitly enabled live services for network boundaries. ~12,341 infrastructure tests and 6293+ project tests are discovered by the introspection build.

`template/`: An Integrated Solution II

3. **Automated Provenance**: Steganographic watermarking and cryptographic hashing are integrated directly into the rendering pipeline. Every generated PDF carries SHA-256 (and, when enabled, SHA-512) fingerprints, an alpha-channel watermark (configurable text or QR overlay), per-page provenance footers and barcode strips, and embedded PDF metadata recording the build timestamp, document identifier, and Git commit hash. Provenance is not asserted by policy; it is enforced by architecture.

4. **AI-Agent Collaboration and Skill-Based Agentic Operations**: A three-tier documentation architecture enables AI agents to operate at every level of the system. At the system level, `CLAUDE.md` asserts global architectural constraints. At the structural level, `AGENTS.md` files at every directory expose local API surfaces, file inventories, and integration contracts. At the module level, `SKILL.md` files—written to a discoverable YAML+Markdown schema aligned with the Model Context Protocol (Anthropic 2024)—define each infrastructure module as a reusable, self-describing tool. An agent invoking `infrastructure.rendering` does not need to read source code: it reads the `rendering/SKILL.md`, which declares the module's name, description, key imports, and example invocations in a machine-parseable YAML frontmatter block. This architecture is the practical realization of the skill-library paradigm established in the agent literature: Yao et al.'s ReAct framework (Yao et al. 2023) demonstrated that interleaving reasoning traces with tool invocations dramatically improves LLM reliability; Schick et al.'s Toolformer (Schick et al. 2023) showed that self-supervised tool use can be bootstrapped from natural language; Wang et al.'s Voyager (Wang et al. 2023) proved that growing skill libraries enable open-ended autonomous exploration in complex environments. `template/` instantiates this vision in the domain of scientific research infrastructure: each `SKILL.md` is a Voyager-style skill, each pipeline stage is a ReAct action, and the full infrastructure layer constitutes a composable,

Scope and Contributions

Scope and Contributions I

This paper is itself a product of the template it describes. The metrics populating its tables were computed by the introspection module documented in the Methods; the figures were rendered by the visualization code validated by the test suite described in Quality Assurance; the PDF carrying these words was assembled by the same YAML-declared pipeline whose architecture is the subject of the Results. This self-productive loop—where the system that is described is also the system that produces the description—is not incidental but structural, a concrete demonstration that `template/` can sustain the full lifecycle from source code to published artifact within a single, version-controlled, pipeline-enforced repository. Our contributions are:

Scope and Contributions I

- A formal description of the Two-Layer Architecture and Standalone Project Paradigm that enables N independent research projects to share infrastructure without coupling.
- A detailed specification of the 17-stage DAG in `infrastructure/core/pipeline/pipeline.yaml`: default full runs use 10 stages; `--mode core` runs 8; opt-in ebook, docxplus, metadata, bundle, and archival stages via `--tags`.
- A comparative analysis positioning `template/` against nine peer tools—Snakemake, Nextflow, CWL, Quarto, Jupyter Book, R Markdown, DVC, Overleaf, OpenAI Prism—across fourteen feature dimensions, demonstrating that `template/` uniquely bundles the eleven capabilities it supports (testing, provenance, documentation, and orchestration enforcement) of the fourteen surveyed dimensions enumerated in §Results.

Scope and Contributions II

- An empirical evaluation across the canonical exemplar projects under `projects/templates/` (`templates/template_active_inference`, `templates/template_advanced_literature_review`, `templates/template_autopoiesis`, `templates/template_autoresearch_project`, `templates/template_autoscientists`, `templates/template_code_project`, `templates/template_data_descriptor`, `templates/template_eda_notebook`, `templates/template_experiment_tree`, `templates/template_formal`, `templates/template_gold_refinement`, `templates/template_literature_meta_analysis`, `templates/template_madlib`, `templates/template_methods_paper`, `templates/template_newspaper`, `templates/template_pitch_deck`, `templates/template_pools_rules_tools`, `templates/template_prose_project`, `templates/template_redacted_report`, `templates/template_registered_report`, `templates/template_search_project`, `templates/template_sia`, `templates/template_storybook`, `templates/template_template`, `templates/template_textbook`)—including this meta manuscript from `projects/templates/template_template`, which exercises introspection-derived metrics.
- A security analysis of the steganographic provenance layer, including a formal threat model and tamper-detection capabilities aligned with the W3C PROV data model (Moreau and Missier 2013) and SLSA (Open Source Security Foundation 2023).

Scope and Contributions III

- An open-source reference implementation available at `github.com/docxology/template`.

Paper Organization

Paper Organization I

The Methods describe the Two-Layer Architecture, Thin Orchestrator pattern, pipeline stages, and AI collaboration model. Results present quantitative metrics from multi-project execution, coverage analysis, and steganographic benchmarks. The Discussion addresses the Zero-Mock tradeoff, scalability implications, a detailed tool comparison, and future directions. The Infrastructure Module Reference provides detailed documentation for all 31 subdirectories. Security and Provenance describes the steganographic and cryptographic integrity layer. The Appendices provide pipeline, configuration, and comparative references.